Tableau et chaine

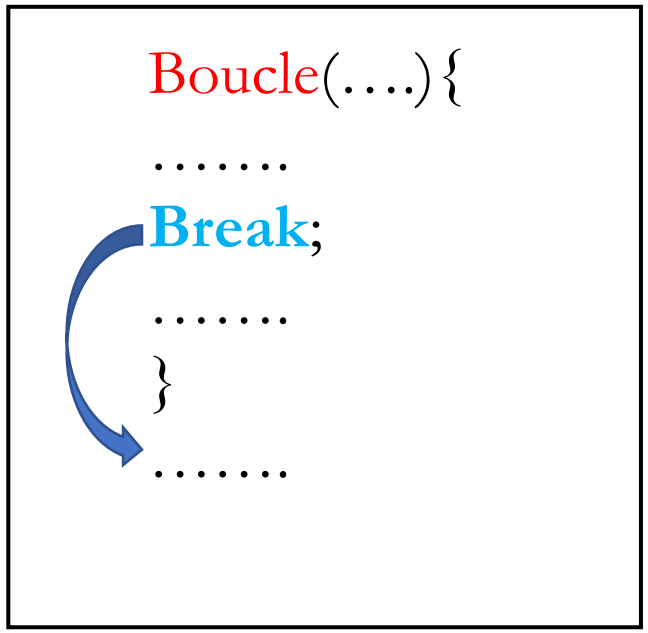
Arfaoui Ahlem

Les structures répétitives:

L'instruction break:

- L'instruction **break** permet d' arrêter le déroulement d'une boucle, et de passer à l'instruction qui suit cette boucle.

Syntaxe:



Exemple:

Ecrire un programme qui calcule la somme d'un maximum de 8 nombres entrés par l'utilisateur, si un nombre négative est entré, la boucle se termine.

Les structures répétitives:

L'instruction break

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int s,n,i;
   s=0;
   for(i=0; i<=8; i++){
      printf("entrer n%d:  ",i);
      scanf("%d",&n);
      if(n<0)
         break;
      s+=n;
   }
printf("La somme est:%d",s);
return 0;
}
```



Affichage:

```
entrer n0:  3
entrer n1:  7
entrer n2:  9
entrer n3:  -4
La somme est:19
Process returned 0 (0x0)   execution time : 13.136 s
Press any key to continue.
```

Les structures répétitives:

L'instruction continue

- L'instruction **continue** permet d' ignorer l'itération actuelle de la boucle et de passer à l'itération suivante.

Syntaxe:

```
Boucle(….){
…….
continue;
…….// l'itération
actuelle
}
…….
```

Exemple:

Ecrire un programme qui calcule la somme d'un maximum de 8 nombres entrés par l'utilisateur, si un nombre négative est entré, la boucle ignore ce nombre.

Les structures répétitives:

L'instruction continue:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int s,n,i;
   s=0;
   for(i=0; i<=8; i++){
      printf("entrer n%d:  ",i);
      scanf("%d",&n);
      if(n<0)
         continue;
      s+=n;
   }
printf("La somme est:%d",s);
return 0;
}
```



Affichage:

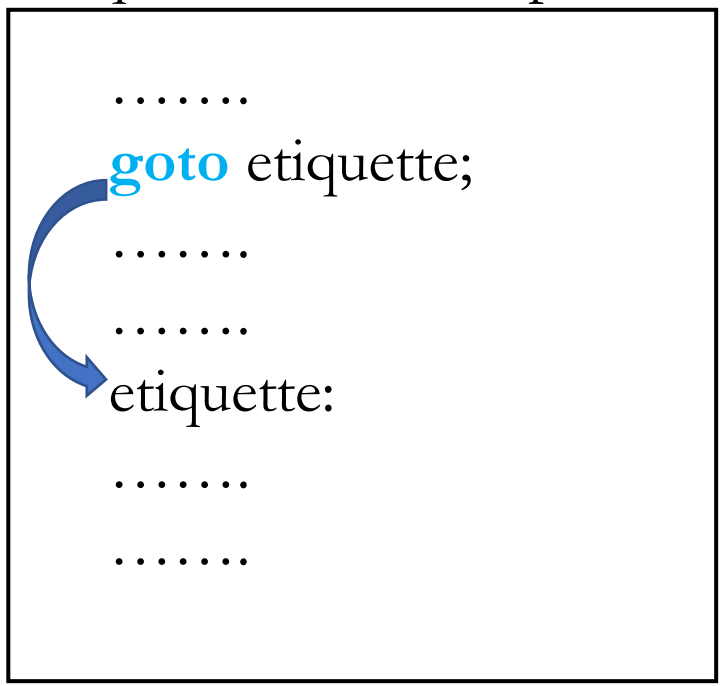
```
entrer n0:  4
entrer n1:  6
entrer n2:  -2
entrer n3:  -4
entrer n4:  8
entrer n5:  9
entrer n6:  -1
entrer n7:  77
entrer n8:  -14
La somme est:104
Process returned 0 (0x0)   execution time : 25.247 s
Press any key to continue.
```

Les structures répétitives:

L'instruction goto:

- L'instruction **goto** permet de se repositionner sur une autre section de code à exécuter introduite par une étiquette, au lieu de poursuivre une exécution séquentielle.

Syntaxe:



Exemple:

Sans utiliser de boucle, écrivez un programme qui demande un nombre entre 1 et 5, jusqu'à ce que la réponse soit appropriée.

Les structures répétitives:

L'instruction goto:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
 int n;
 debut:
 printf("entrer un nombre entre 1
et 5 ");
  scanf("%d",&n);
  if(n<1 || n>5)
      goto debut;
  printf("C'est correcte");
return 0;
}
```



Affichage:

```
entrer un nombre entre 1 et 5 7
entrer un nombre entre 1 et 5 22
entrer un nombre entre 1 et 5 -2
entrer un nombre entre 1 et 5 4
C'est correcte
Process returned 0 (0x0)   execution time : 23.671 s
Press any key to continue.
```

Les tableaux(1):

- **Introduction:**

Problème:

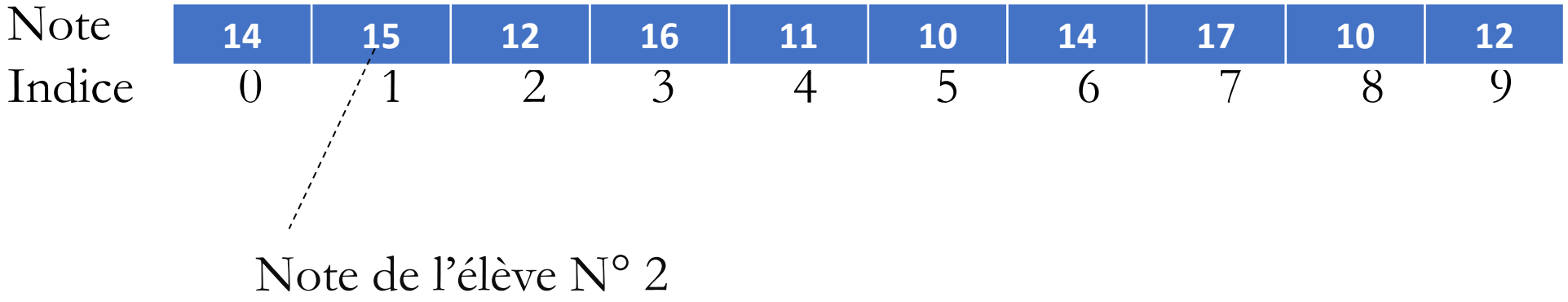
- Pour conserver par exemple simultanément les notes de 10 élèves, il nous faut de 10 variables différentes (et si on avait 100 ou 1000 élèves ???).

Inconvénients:

- Un nom pour chaque variable.
- Aucun lien entre les différentes variables.

Solution:

- Disposer d'un objet plus complexe, pour stocker ces notes, et y accéder à l'aide d'un indice.



Les tableaux(2):

Introduction:

Intérêts:

- Gain de temps.
- Rétrécissement du volume du programme.
- Possibilité de réutilisation de toutes les valeurs ultérieurement dans le programme.

Définition:

- Un tableau est une structure de donnée permettant de **stocker des données de même type**.
- Chacune des valeurs est repérée par un indice indiquant la position de la donnée dans le tableau.
- Un tableau est **caractérisé** par :
  - ❑ Son nom .
  - ❑ Sa taille .
  - ❑ Sa dimension .
  - ❑ Le type de ses éléments.

Les tableaux(3):

Les tableaux à une dimension (uni-dimensionnel):

- Un tableau uni-dimensionnel (**vecteur**) est une manière de ranger des éléments ou des valeurs de **même type**.
- Il regroupe ces éléments dans **une structure fixe** et permet d'accéder à chaque élément par l'intermédiaire de son rang ou indice.

Tableau A:



Déclaration:

Syntaxe: **<type simple> <Nom_tableau> [<taille>];**

Exemple:
```
int A [25];
char nom[30];
float B[100];
```

Les tableaux(4):

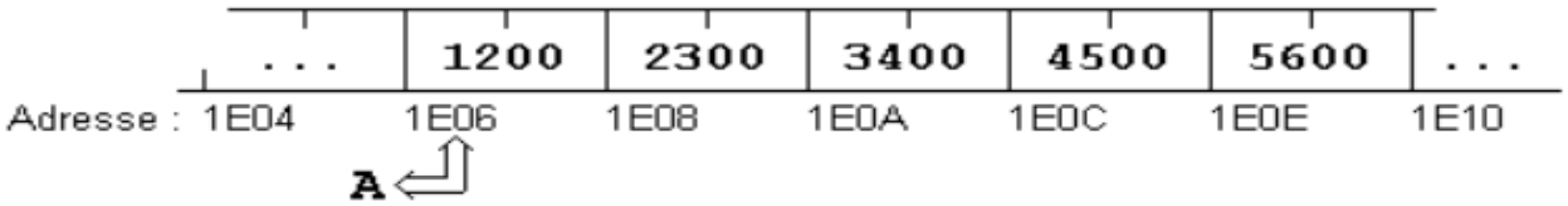
Les tableaux à une dimension (uni-dimensionnel):

Déclaration:

- ✓ <**taille**> est nécessairement une valeur numérique.
- ✓ Pour un tableau de taille N, le compilateur C réserve N places en mémoire pour ranger les éléments du tableau.
- ✓ Un élément du tableau est repéré par son indice. En langage C les tableaux commencent à **l'indice 0**. L'indice **maximum** est donc **N-1**.
- ✓ En langage C, le nom d'un tableau est le représentant de l'adresse du premier élément du tableau
- ✓ Les adresses des autres composantes sont calculées (automatiquement) relativement à cette adresse

Exemple:

```
short A[5] = {1200, 2300, 3400, 4500, 5600};
```



Les tableaux(5):

Les tableaux à une dimension (uni-dimensionnel):

Déclaration:

✓ Espace occupé en mémoire: **taille d'une composante en octets× *N*.**

Exemple:

- short A[4]; //taille =2 octets × 4
- float tab[10]; // taille =4 octets × 10

Initialisation:

Lors de la déclaration d'un tableau, on peut initialiser les composantes du tableau en indiquant la liste des valeurs respectivement entre {}

Les tableaux(6):

Les tableaux à une dimension (uni-dimensionnel):

Initialisation:

Exemple:

- Short A[5]={1,10,9,4,5};

=> Réservation de (2×5) octets.

- Char lettre[26]={'A','B'};

=> Les autres composantes initialisées à 0.

- Float B[]={2.4, 1.6, 3.33};

=> Réservation automatique de (4×3) octets (la dimension n'est pas indiquée explicitement).

- Short A[4]={1,10,9,4,5};

=> Erreur!

Les tableaux(7):

Les tableaux à une dimension (uni-dimensionnel):

Affectation:

Syntaxe:

<Nom_tableau>[<indice>]= <expression>;

Exemple:

T[2]=3;

Remplir un tableau:

Remplir un tableau à **l'aide d'une boucle**

```
int T [4], i, x;
 for (i= 0; i< 4; i++)
{
printf ("Donnez l'élément n°:%d", i);
scanf (“%d”, &x);
 T[i] = x;
}
```

Les tableaux(8):

Les tableaux à une dimension (uni-dimensionnel):

Remplir un tableau:

On peut initialiser un tableau lors **de sa déclaration**

int Tpremiers [4] = {2, 3, 5, 7};

Et même: int Tpremiers [] = {2, 3, 5, 7};

Attention:

- **on ne peut pas utiliser l'affectation !**

~~int Tpremiers [4];~~

~~Tpremiers = {2, 3, 5, 7};~~

- **On ne peut pas affecter un tableau à un autre par =**

~~int T1 [4] = {2, 3, 5, 7};~~

~~int T2 [4];~~

~~T2 = T1;~~

Les tableaux(9):

Les tableaux à une dimension (uni-dimensionnel):

Attention:

❖ **Ni comparer un tableau avec un autre par ==**

```
int T1 [4] = {2, 3, 5, 7};
 int T2 [4] = {2, 3, 5, 7};
if (T1 == T2)
printf ("Tableaux identiques.");
else
printf ("Tableaux différents.");
```

Les tableaux(10):

Les tableaux à une dimension (uni-dimensionnel):

Exemple1:

Ecrire un programme permettant de saisir le nombre d'éléments d'un tableau d'entiers le remplit, puis l'affiche.

Les tableaux(11):

Les tableaux à une dimension (uni-dimensionnel):

Solution:

```
int main()
{
   int Tab[50];
   int i,n;
   printf("saisir le nombre  d'elements du tableau
\n");
   scanf("%d",&n);
   for(i=0;i<n;i++)
      {
         printf("saisir un element:\n");
         scanf("%d",&Tab[i]);
      }
   printf("affichage du tableau:\n");
   for(i=0;i<n;i++)
      printf("%d ", Tab[i]);
   return 0;
}
```



```
saisir le nombre  d'elements du tableau
5
saisir un element:
7
saisir un element:
6
saisir un element:
4
saisir un element:
3
saisir un element:
2
affichage du tableau:
7 6 4 3 2
Process returned 0 (0x0)   execution time : 15.849 s
Press any key to continue.
```

Les tableaux(12):

Les tableaux à une dimension (uni-dimensionnel):

Exemple2:

Ecrire un algorithme qui détermine la plus grande et la plus petite valeur dans un tableau d'entiers A. Afficher ensuite la valeur et la position du maximum et du minimum. Si le tableau contient plusieurs maxima ou minima, le programme retiendra la position du premier maximum ou minimum rencontré.

Les tableaux(13):

Les tableaux à une dimension (uni-dimensionnel):

Solution:

```
#include<stdio.h>
void main()
{
   int n=5,i=0,t[n];
   t[0]=3;
   t[1]=1;
   t[2]=1;
   t[3]=5;
   t[4]=5;
   int imax=-1,imin=-1,max=t[0];
   int min=t[0];
   while(i<n)
   {printf("%d | ",t[i]);
   i++;}
```

```
printf("\n");
   i=0;
   while(i<n)

   {
      if (t[i]>max){ max=t[i]; imax=i;}
      if(t[i]<min) {min=t[i]; imin=i;}
      i++;
   }

printf("le max est %d d indice %d\n",max,imax);

printf("le min est %d d indice %d \n",min,imin);

}
```

Les tableaux(14):

Les tableaux à deux dimensions (Matrice):

Définition:

- En C, un tableau à deux dimensions A est à interpréter comme un tableau (uni-dimensionnel) de dimension L dont chaque composante est un tableau (uni-dimensionnel) de dimension C.
- On appelle L le ***nombre de lignes*** du tableau et C le ***nombre de colonnes*** du tableau. L et C sont alors les deux ***dimensions*** du tableau. => Un tableau à deux dimensions contient donc ***L*C composantes***.

L Lignes

		.....	
		.....	
.....	.....	.....	.....
		.....	

C Colonnes

Les tableaux(15):

Les tableaux à deux dimensions (Matrice):

Déclaration:

<TypeSimple> <NomTabl>[<tailleLigne>][<tailleCol>];

Exemples:

- int Notes [3][20]; /* 20 étudiants, 3 matières */
- char T[12][18];
- float A[2][2];

Les composantes d'un tableau à 2 dimensions sont stockés ligne par ligne dans la mémoire

Espace occupé en mémoire: => Taille d'une composante en octets × L × C

Exemple:

short A [4][5];/* $Taille = 2\,octets \times 4 \times 5$ */

Les tableaux(16):

Les tableaux à deux dimensions (Matrice):

Initialisation:

- Lors de la déclaration d'un tableau, on peut initialiser les composantes du tableau en indiquant la liste des valeurs respectivement entre **{}.**
- A l 'intérieur de la liste, les composantes de chaque ligne sont entre **{}.**

Exemples:

- short A [3][5] = { {1, 10, 9, 4, 5}, {1, 1, 3, 4, 5},{1, 10, 3, 4, 4}};
  => Réservation de (2 x 3 x 5) octets
- int C [4][4] = {{1, 1, 1, 1}};
  => Les autres composantes initialisées à 0
- float B [ ][3] = { {2.4, 1.6, 3.33},{0.5, 0.2, 0.1}} ;
  => Réservation automatique de (4x2 x3) octets (Pour les lignes seulement!)
- int C [4][4] = {{1, 1, 1, 1, 1}};
  => Erreur!

Les tableaux(17):

Les tableaux à deux dimensions (Matrice):

Accès:

Syntaxe: **<Nom_tableau>[<lignes>][<colonne>]**

Exemple:

Pour un tableau T de taille L et C :

- ✓ **T[0][0]** pour accéder **au premier** élément
- ✓ **T[L-1][C-1]** pour accéder **au dernier** élément

Affectation:

Syntaxe: **<Nom_tableau>[<lignes>][<colonne>]=<expression>;**

Exemple:

```
int T[0][1] = 2;
int Notes [5][100];
```

Les chaines de caractères(1):

- Il n'existe pas de type spécial **chaîne** ou **string** en C. Une chaîne de caractères est traitée comme un **tableau à une dimension de caractères** (vecteur de caractères). ➔Il existe des notations particulières et une bonne quantité de fonctions spéciales pour le traitement de tableaux de caractères.

- Le **principe** est le suivant, dans un tableau suffisamment grand, on stocke à la suite les différents caractères de la chaîne. A la fin de la chaîne, on rajoute un caractère supplémentaire qui fait office **de marqueur** de fin de chaîne. C'est le caractère dont le code **ASCII** est **zéro**, que l'on note **'\0'**. **Attention** toutefois à ne pas le confondre avec le caractère '0' dont le code ASCII est 48!

- Il faut toujours se souvenir qu'un tableau de **N** positions ne peut contenir au maximum qu'une chaîne de longueur **N-1**, **le dernier élément du tableau devant forcément contenir le marqueur de fin de chaîne.**

Les chaines de caractères(2):

Déclaration:

char <NomChaine> [<Longueur>];

Exemple:

- char NOM [20];
- char PRENOM [20];
- char PHRASE [300];

Espace à réserver:

✓ Lors de la déclaration, nous devons **indiquer l'espace à réserver en mémoire** pour le stockage de la chaîne.

✓ La représentation interne d'une chaîne de caractères est terminée par le symbole **'\0'** (NUL). Ainsi, pour un texte de **n** caractères, nous devons prévoir **n+1 octets**.

✓ Malheureusement, le compilateur C **ne contrôle pas si nous avons réservé un octet pour le symbole de fin de chaîne**. => l'erreur se fera seulement remarquer lors de l'exécution du programme

Les chaines de caractères(3):

- ✓ Le nom d'une chaîne représente l'adresse du premier caractère de la chaîne.
- ✓ utilisation du format **%s** pour les chaines de caractères.
- ✓ Comme le nom d'une chaîne de caractères est le représentant de l'adresse du premier caractère de la chaîne, il ne doit pas être précédé de l'opérateur adresse '**&**'.

Exemple:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{   char nom [20];
    printf("saisir un
nom");
    scanf("%s",nom);
    printf("%s", nom);
    return 0;
}
```



```
saisir un nom Ahmed
Ahmed
Process returned 0 (0x0)   execution time : 16.314 s
Press any key to continue.
```

Les chaines de caractères(4):

Remarque:

Scanf(”%s”,nom); ou **scanf(“%s”, nom+ 2);** => avec scanf, il faut donner une adresse mémoire à partir de laquelle on va placer la chaine lue.

printf(”%s”,nom); ou **printf(“%s”, nom+2);** => avec printf, il faut donner aussi une adresse mémoire à partir de laquelle on va afficher la chaine.

Exemple:

```
#include <stdio.h>
#include <stdlib.h>
int main()
{   char nom [50];
    printf("saisir un nom\n");
    scanf("%s",nom);
    printf(" le nom est: %s \n", nom);
    printf("le nom est: %s \n", nom+2);
    printf("resaisir une autre fois\n");
    scanf("%s", nom+2);
    printf(" le nom est: %s \n", nom);
    printf("le nom est: %s \n", nom+2);
    return 0;
}
```



```
saisir un nom
MOUHAMED
 le nom est: MOUHAMED
le nom est: UHAMED
resaisir une autre fois
BBBBB
 le nom est: MOBBBBB
le nom est: BBBBB

Process returned 0 (0x0)   execution time : 30.012 s
Press any key to continue.
```

Les chaines de caractères(5):

Initialisation de chaînes de caractères:

- ✓ En général, les tableaux sont initialisés par l'indication de la liste des éléments du tableau entre **accolades**:
  - **char Caine[] = {'H','e','l','l','o','\0'};**
- ✓ Pour le cas spécial des tableaux de caractères, nous pouvons utiliser une initialisation plus confortable en indiquant simplement une chaîne de caractère constante:
  - **char chaine[] = "Hello";**
- ✓ Lors de l'initialisation par **[]**, l'ordinateur réserve automatiquement le nombre d'octets nécessaires pour la chaîne, c.-à-d.: **le nombre de caractères + 1** (ici: 6 octets). Nous pouvons **aussi indiquer explicitement le nombre d'octets à réserver**, si celui-ci est supérieur ou égal à la longueur de la chaîne d'initialisation.

Les chaines de caractères(6):

Exemples:

1. Char text[]=“ Hello”;



=> correct

2. Char text[6]=“ Hello”;



=> correct

3. Char text[8]=“ Hello”;



=> correct

4. Char text[5]=“ Hello”;



=>Erreur pendant l’exécution

Les chaines de caractères(7):

Accès aux éléments d'une chaîne:

L'accès à un élément d'une chaîne de caractères peut se faire de la même façon que l'accès à un élément d'un tableau. En déclarant une chaîne par:

char A[6];

Nous avons défini un tableau A avec six éléments, auxquels on peut accéder par:

A[0], A[1], ... , A[5]

Travailler avec des chaînes de caractères:

Les bibliothèques de fonctions de C contiennent une série de fonctions spéciales pour le traitement de chaînes de caractères.

- **Les fonctions de <stdio.h>:**

✓ Comme nous l'avons déjà vu, la bibliothèque **<stdio>** nous offre des fonctions qui effectuent l'entrée et la sortie des données. A côté des fonctions **printf** et **scanf** que nous connaissons déjà, nous y trouvons les deux fonctions **puts** et **gets,** spécialement conçues pour l'écriture et la lecture de chaînes de caractères.

Les chaines de caractères(8):

- **Les fonctions de <stdio.h>:**

- Affichage de chaînes de caractères:

1) **printf** avec le spécificateur de format **%s** permet d'intégrer une chaîne de caractères dans une phrase.
2) **puts** est idéale pour écrire une chaîne constante ou le contenu d'une variable dans une ligne isolée.

Syntaxe: **puts(<Chaîne>)**

Effet: **puts** écrit la chaîne de caractères désignée par <Chaîne> sur **stdout** et provoque un retour à la ligne. En pratique

puts(txt); est équivalent à **printf("%s\n",txt);**

Exemples

```
char TEXTE[] = "Voici une première ligne.";
puts(TEXTE);
puts("Voici une deuxième ligne.");
```

Les chaines de caractères(9):

- **Les fonctions de <stdio.h>:**

- **Lecture de chaînes de caractères:**

1) scanf avec le spécificateur **%s** permet de lire un mot isolé à l'intérieur d'une suite de données du même ou d'un autre type.

Exemple

```
char lieu[25];
int jour, moins, annee;
printf("Entrez lieu et date de naissance : \n");
scanf("%s %d %d %d", lieu, &jour, &moins, &annee);
```

Effet: scanf avec le spécificateur **%s** lit un **mot** du fichier d'entrée standard **stdin** et le mémorise à l'adresse qui est associée à **%s**.

2) **gets** est idéal pour lire une ou plusieurs lignes de texte (p.ex. des phrases) terminées par un retour à la ligne.

Syntaxe: gets(<Chaîne**>)**

Effet: gets lit une ***ligne*** de de caractères de **stdin** et la copie à l'adresse indiquée par <Chaîne>. Le retour à la ligne final est remplacé par le symbole de fin de chaîne **'\0'**.

Les chaines de caractères(10):

- **Les fonctions de <string.h>:**

✓ La bibliothèque <**string**> fournit une multitude de fonctions pratiques pour le traitement de chaînes de caractères.

- **Fonctions pour le traitement de chaînes de caractères:**

✓ **strlen(<s>):** fournit la longueur de la chaîne ***sans*** compter le '\0' final
strcpy(<s>, <t>): copie <t> vers <s>
strcat(<s>, <t>): ajoute <t> à la fin de <s>

✓ **strcmp(<s>, <t>)**: compare <s> et <t> lexicographiquement et fournit un résultat:

- **Négatif** si <s> précède <t>
- **zéro** si <s> est égal à <t>
- **positif** si <s> suit <t>

✓ **strncat(<s>, <t>, <n>)**: ajoute au plus <n> caractères de <t> à la fin de <s>

Les chaines de caractères(11):

- **Les fonctions de <string.h>:**

Remarques:

- Comme le nom d'une chaîne de caractères représente **une adresse fixe en mémoire**, on ne peut pas '**affecter**' une autre chaîne au nom d'un tableau:

~~A="Hello"~~

=> Il faut bien copier la chaîne caractère par caractère ou utiliser la fonction **strcpy** respectivement **strncpy**:

strcpy(A, "Hello");

Les chaines de caractères(12):

Exemples:

- **Strcat et strncat:**

```
#include <string.h>
Int main ()
{
char ch1[50]="bonjour" ;
char ch2[]=" monsieur";

printf ("avant : %s\n ", ch1);
strcat (ch1, ch2);
printf ("apres : %s\n",ch1);
strncat (ch1,ch2,4);
printf ("apres : %s\n",ch1);
}
```



```
avant : bonjour
 apres : bonjour monsieur
apres : bonjour monsieur mon

Process returned 0 (0x0)   execution time : 0.030 s
Press any key to continue.
```

Les chaines de caractères(13):

Exemples:

Strlen:

```
#include <string.h>
Int main ()
{
char CH1 [] = "hello" ;
char CH2 [] = " bonjour " ;

printf("%d",strlen (CH1)) ;
}
```



=> **Il affiche 5**

Strcpy

```
#include <string.h>
main ()
{
char CH1 [] = "hello" ;
char CH2 [] = "bonjour" ;
printf ("avant : %s\n ", CH1);
strcpy (CH1, CH2) ;
printf ("apres : %s\n ", CH1);
}
```



```
avant : hello
 apres : bonjour
```

Les chaines de caractères(14):

Exemples:

Strcmp:

```
#include <string.h>
main ()
{
char CH1 [] = "hello" ;
char CH2 [] = "bonjour" ;
strcmp (CH1, CH2) ;
printf ("% s\n ", strcmp (CH1,
CH2));
 return 0;
}
```

```
Process returned -1073741819 (0xC0000005)   execution time : 2.282 s
Press any key to continue.
```

```
#include <string.h>
main ()
{
char CH1 [] = "hello" ;
char CH2 [] = « hello" ;
strcmp (CH1, CH2) ;
printf ("% s\n ", strcmp (CH1,
CH2));
 return 0;
}
```

```
(null)

Process returned 0 (0x0)   execution time : 0.038 s
Press any key to continue.
```

Les chaines de caractères(15):

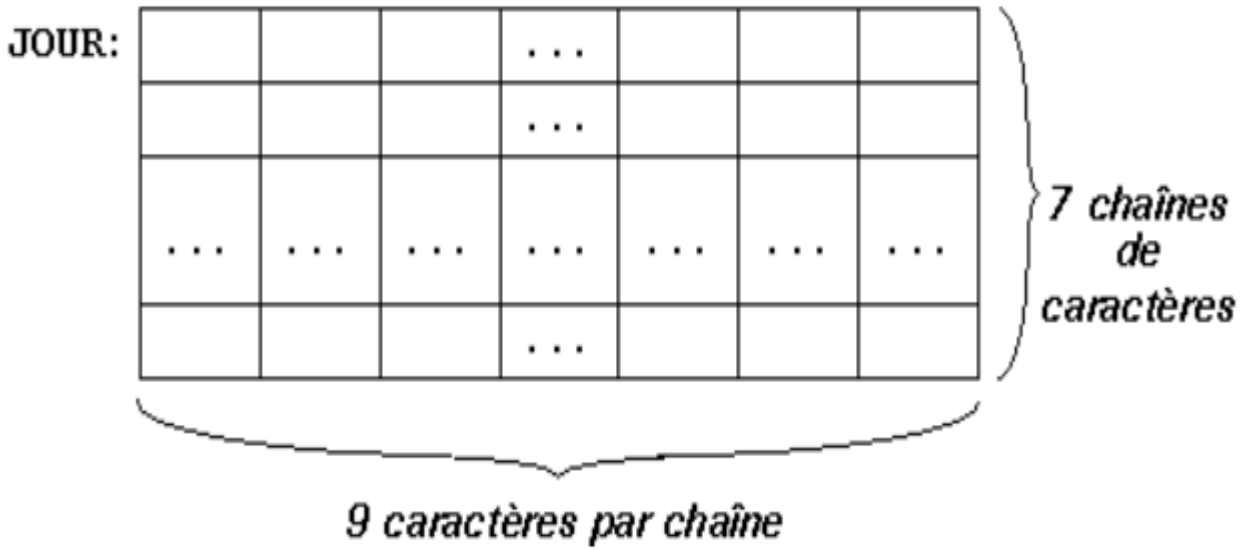
Tableaux de chaînes de caractères:

Un tableau de chaînes de caractères correspond à un tableau à deux dimensions du type **char**, où **chaque ligne contient une chaîne de caractères**.

Déclaration:

La déclaration **char jour[7][9];**

⇒ réserve l'espace en mémoire pour 7 mots contenant 9 caractères (dont 8 caractères significatifs).



Les chaines de caractères(16):

Tableaux de chaînes de caractères:

Initialisation:

Lors de la déclaration il est possible d'initialiser toutes les composantes du tableau par des chaînes de caractères constantes:

char jour[7][9]= {"lundi", "mardi", "mercredi", "jeudi", "vendredi", "samedi", "dimanche"};

JOUR:									
	'l'	'u'	'n'	'd'	'i'	'\0'			
	'm'	'a'	'r'	'd'	'i'	'\0'			
	'm'	'e'	'r'	'c'	'r'	'e'	'd'	'i'	'\0'
	...	...	...	...	...	...	...	...	...
	'd'	'i'	'm'	'a'	'n'	'c'	'h'	'e'	'\0'

➔ Les tableaux de chaînes sont mémorisés ligne par ligne. La variable jour aura donc besoin de **7*9*1 = 63 octets en mémoire**.

Les tableaux(1):

Les tableaux à deux dimensions (Matrice):

Exercice1:

Ecrire un programme qui lit les dimensions L et C d'un tableau T à deux dimensions du type **int** (dimensions maximales: 50 lignes et 50 colonnes). Remplir le tableau par des valeurs entrées au clavier et afficher le tableau ainsi que la somme de chaque ligne et de chaque colonne en n'utilisant qu'une variable d'aide pour la somme.

Les tableaux(2):

Les tableaux à deux dimensions (Matrice):

Exercice2:

Ecrire un programme qui transfère un tableau M à deux dimensions L et C (dimensions maximales: 10 lignes et 10 colonnes) dans un tableau V à une dimension L*C.

Exemple:

a	b	c	d
e	f	g	h
i	j	k	l



a	b	c	d	e	f	g	h	i	j	k	l

Les tableaux(3):

Les tableaux à deux dimensions (Matrice):

Exercice 3:

Ecrire un programme C qui multiplie une matrice carré d'entiers M(N*N) par un réel X donné.

- Les éléments de la matrice A seront multipliés par X.
- Le résultat de la multiplication sera mémorisé dans une deuxième matrice A qui sera ensuite affichée.

Les chaines de caractères(1):

Exercice 1:

Ecrire un programme qui lit une chaîne de caractères CH et qui convertit toutes les majuscules dans des minuscules et vice-versa.

Le résultat sera mémorisé dans la même variable CH et affiché après la conversion.

Les chaines de caractères(2):

Exercice 2:

Ecrire programme C qui permet de tester si un mot est un palindrome ou non. Un mot est dit palindrome s'il se lit de la même manière de gauche à droite ou de droite à gauche.

Exemple : LAVAL, AZIZA, AFIFA.

Les chaines de caractères(3):

Exercice 3:

Ecrire un programme qui calcule la distance de Hamming entre deux mots entrés par l'utilisateur.

NB:

La distance de Hamming entre deux mots de même longueur est le nombre d'endroits ou les lettres sont différentes.

Exemple:

JAPON – SAVON ➔ la distance de Hamming est 2